HDOC DAY – 66

Algorithm

1. Start.
2. Read the order n of the square matrices.
3. Read the elements of matrices A and B.
4. Initialize all elements of product matrix C to 0.
5. Call the recursive function multiply() with i = 0, j = 0, and k = 0.
6. If i >= n, terminate the recursion.
7. If j >= n, move to the next row by calling the function with i + 1 and j = 0.
8. If k < n, calculate A[i][k] * B[k][j] and add it to C[i][j].
9. Recursively increase k until one element of the product matrix is calculated.
10. Move to the next column and repeat the recursive process.
11. Display the product matrix.
12. Stop.

C PROGRAM:

```
#include <stdio.h>


#define MAX 10


void multiply(int A[MAX][MAX], int B[MAX][MAX],
        int C[MAX][MAX], int n, int i, int j, int k)
{
   if (i >= n)
```

```
        return;

    if (j >= n)
    {
        multiply(A, B, C, n, i + 1, 0, 0);
        return;
    }

    if (k < n)
    {
        C[i][j] = C[i][j] + A[i][k] * B[k][j];
        multiply(A, B, C, n, i, j, k + 1);
    }
    else
    {
        multiply(A, B, C, n, i, j + 1, 0);
    }
}

int main()
{
    int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX] = {0};
    int n, i, j;

    printf("Enter the order of square matrices: ");
```

```
scanf("%d", &n);

printf("Enter elements of matrix A:\n");
for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
      scanf("%d", &A[i][j]);
}

printf("Enter elements of matrix B:\n");
for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
      scanf("%d", &B[i][j]);
}

multiply(A, B, C, n, 0, 0, 0);

printf("Product matrix:\n");
for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
      printf("%d ", C[i][j]);

   printf("\n");
```

```
    }


    return 0;
}
```

TEST CASE TABLE:

Test Case	Matrix A	Matrix B	Expected Product
1	1 2 `3 4`	5 6 `7 8`	19 22 `43 50`
2	1 0 `0 1`	2 3 `4 5`	2 3 `4 5`
3	2 0 `0 2`	1 2 `3 4`	2 4 `6 8`
4	5	3	15